

PROJECT DOCUMENTATION V1.0

Expense Tracker Web App

A full-featured personal finance tracker with cloud sync, user authentication, budget management, and cross-device access — built as a Progressive Web App.

VERSION	PLATFORM	BACKEND	HOSTING
Advanced Edition	Web PWA + Android	Supabase (PostgreSQL)	GitHub Pages (Free)

- HTML5
- CSS3
- Vanilla JavaScript
- Supabase Auth
- PostgreSQL
- Chart.js
- jsPDF
- PWA



Table of Contents

1	Project Overview What it is & what it does	2	Features Complete feature list
3	Architecture How the system is structured	4	Supabase Setup Database & auth configuration
5	Project File Structure Files and their roles	6	Code Walkthrough Every block explained simply
7	UI Screens Each screen explained	8	Data Flow How data moves through the app
9	Deployment Guide GitHub Pages + PWA install	10	Troubleshooting Common issues & fixes
11	Future Enhancements Ideas for next version	12	Glossary Technical terms explained

SECTION 01

Project Overview

The Expense Tracker is a modern personal finance web application that lets users record, categorize, visualize, and manage their daily expenses. It is designed as a Progressive Web App (PWA), which means it can be installed on Android phones and works like a native app — even offline.

What Problem Does It Solve?

Most people lose track of where their money goes each month. This app gives a simple, fast way to log every expense on your phone, see visual breakdowns by category, set monthly budgets, and get alerts when you are overspending — all without paying for any subscription or app.

Technology Stack (Simple Terms)

TECHNOLOGY	WHAT IT IS	WHY USED
HTML	The skeleton of every web page	Defines all buttons, forms, and text on screen
CSS	The styling language	Makes everything look dark, modern, and mobile-friendly
JavaScript	The programming language of browsers	Makes the app interactive — saves data, shows charts, handles login
Supabase	A free cloud database service	Stores all expense data online so it loads on any device
Chart.js	A charting library	Draws the donut chart and bar graph
jsPDF	A PDF generation library	Creates downloadable expense reports

SECTION 02

Features



User Authentication

Sign up and sign in with email and password. Each user sees only their own data. Powered by Supabase Auth.



Cloud Sync

Data is stored in Supabase PostgreSQL database. Opens on any browser or device — even incognito — and shows your data.



Offline Cache

Data is also saved in the device's localStorage so the app loads instantly without waiting for the server.



Add Expenses

Log expenses with name, amount, date, category, and an optional note. Data saves to Supabase instantly.



Edit & Delete

Tap edit on any expense to update it, or delete it. All changes sync to the cloud immediately.



Custom Categories

Add your own categories with a custom colour. Rename or delete them. Set a monthly budget limit per category.



Budget Tracking

Set a monthly spending limit for each category. Progress bars show how much you have used. Alerts fire at 80% and 100%.



Visual Charts

Donut chart shows spending by category. Bar chart shows spending trend over the last 6 months.



Period Filter

Switch between This Month, Last Month, 3 Months, This Year, and All Time views with one tap.



Search & Filter

Search expenses by name, category, or note. Filter by date range and category chip. Results show total instantly.



Export Reports

Download filtered expenses as a CSV file (for Excel) or a formatted PDF report with grand total.



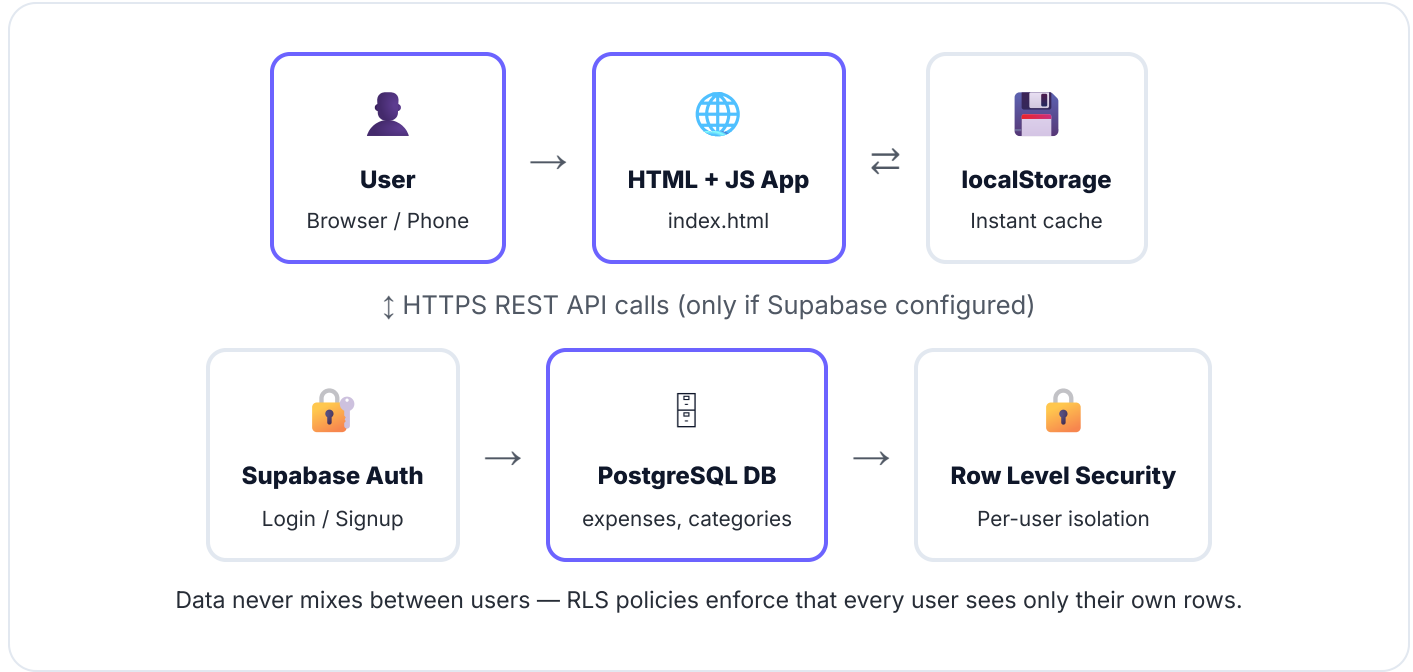
PWA / Installable

Install on Android from Chrome. Opens full screen, no browser bar, like a native app. Works on Nothing Phone.

SECTION 03

System Architecture

The app follows a simple three-layer architecture: the user's browser (frontend), a local cache for speed, and Supabase as the cloud backend.



Load Strategy (Fast First)

When the app opens, it immediately shows data from localStorage (fast, instant). Then in the background it fetches fresh data from Supabase and updates the screen. This is called a **Cache-First, Network-Update** strategy — the user never sees a blank loading screen.

SECTION 04

Supabase Configuration

Supabase is a free, open-source backend-as-a-service. It gives us a PostgreSQL database, user authentication, and a REST API — all for free, with no server to manage.

Step-by-Step Supabase Setup

1 Create a Supabase Account

Go to supabase.com → Sign up (use Google for speed) → Click "New Project" → Give it a name like *expense-tracker* → Set a strong database password → Choose a region close to you → Click "Create Project". Wait about 2 minutes.

2 Open the SQL Editor

In your project dashboard, click

SQL Editor

in the left sidebar. This is where you run commands to create your database tables.

3 Create Tables — Run This SQL

Paste the entire block below into the SQL editor and click

Run

:

SQL

Supabase SQL Editor

```
-- 1. Create the expenses table create table if not exists expenses ( id bigint primary
key generated always as identity, user_id uuid references auth.users(id) on delete
cascade, name text not null, amt numeric not null, cat text not null, date text not null,
note text default '', created_at timestamptz default now() ); -- 2. Create the categories
table create table if not exists categories ( id bigint primary key generated always as
identity, user_id uuid references auth.users(id) on delete cascade, name text not null,
color text not null, budget numeric default 0 ); -- 3. Enable Row Level Security (data
isolation per user) alter table expenses enable row level security; alter table
categories enable row level security; -- 4. Create security policies (each user only sees
their rows) create policy "Users manage own expenses" on expenses for all to
authenticated using (auth.uid() = user_id) with check (auth.uid() = user_id); create
policy "Users manage own categories" on categories for all to authenticated using
(auth.uid() = user_id) with check (auth.uid() = user_id);
```

1 Get Your API Credentials

Go to

Project Settings → API

. Copy two values: *Project URL* (looks like `https://abcdef.supabase.co`) and *anon public key* (a long string starting with `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...`). These are safe to include in your HTML file.

2 Paste Into the App

Open `index.html`. Find the `CONFIG` section near the top of the `<script>` tag and paste your values:

JAVASCRIPTindex.html – CONFIG section

```
const SB_URL = 'https://abcdefgh.supabase.co'; // Your Project URL const SB_KEY = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 ... '; // Your anon key
```



Is it safe to put the key in the HTML?

Yes — the *anon* key is designed to be public. It is not a secret. The Row Level Security policies ensure each user can only access their own data, not others'.

Database Tables Explained

TABLE	COLUMN	TYPE	WHAT IT STORES
expenses	id	bigint	Auto-generated unique number for each expense
	user_id	uuid	Links expense to a specific user
	name	text	Description, e.g. "Grocery run"
	amt	numeric	Amount in rupees, e.g. 2400
	cat	text	Category name, e.g. "Food & Dining"
	date	text	Date as YYYY-MM-DD, e.g. "2026-05-10"
	note	text	Optional extra note
categories	id	bigint	Auto-generated unique number
	user_id	uuid	Links category to specific user
	name	text	Category label, e.g. "Transport"
	color	text	Hex colour, e.g. "#3b82f6"

SECTION 05

Project File Structure

FOLDER STRUCTURE

[GitHub Repository](#)

```
expense-tracker/ |— index.html ← The entire app (HTML + CSS + JS in one file) |—  
manifest.json ← PWA config: app name, icon, theme colour |— README.md ← (Optional)  
Project description on GitHub
```



Why one file?

Keeping everything in `index.html` makes deployment extremely simple — no build tools, no npm, no Node.js. Just upload one file and it works.

manifest.json (for PWA install)

JSON

[manifest.json](#)

```
{ "name": "Expense Tracker", "short_name": "Expenses", "start_url": "/expense-tracker/",  
  "display": "standalone", // hides browser bar – feels native "background_color":  
  "#0f1117", "theme_color": "#0f1117", "icons": [ { "src": "icon-192.png", "sizes":  
  "192×192", "type": "image/png" }, { "src": "icon-512.png", "sizes": "512×512", "type":  
  "image/png" } ] }
```

SECTION 06

Code Walkthrough

The entire app is inside one `index.html` file. It has three parts: HTML (structure), CSS (look), and JavaScript (behaviour). Below is each major JavaScript block explained in plain language.

1. Supabase Client Initialization

This runs once when the page loads. It creates a connection to your Supabase project using your URL and key.

JAVASCRIPT

INIT block

```
// supabase.createClient connects to your cloud database window.addEventListener('load',  
async () => { if (SB_URL && SB_KEY) { // Create the Supabase client – like opening a  
phone line to your database sb = supabase.createClient(SB_URL, SB_KEY); // Watch for  
login/logout events automatically sb.auth.onAuthStateChange((evt, session) => { if (evt  
=== 'SIGNED_IN') { user = session.user; showApp(); } if (evt === 'SIGNED_OUT') {  
showAuthScreen(); } })); } });
```

In simple terms: `onAuthStateChange` is like a listener that Supabase calls automatically whenever the user logs in or logs out. We don't need to manually check — Supabase tells us.

2. Authentication (Login / Signup)

The `doAuth()` function handles both login and signup with the same form. It validates inputs first, then calls Supabase.

JAVASCRIPT

doAuth() function

```
async function doAuth() { const email = document.getElementById('aEmail').value.trim();  
const pass = document.getElementById('aPass').value; // Validation – stop early if inputs  
are wrong if (!email || !pass) { err.textContent = 'Enter email and password'; return; }  
if (pass.length < 6) { err.textContent = 'Password too short'; return; } if (authMode ===  
'login') { // signInWithPassword checks email+password against Supabase Auth const {  
error } = await sb.auth.signInWithPassword({ email, password: pass }); if (error) throw  
error; // goes to catch → shows friendly message } else { // signUp creates a new user  
account in Supabase Auth const { data, error } = await sb.auth.signUp({ email, password:  
pass, options: { data: { name } } } // store the user's name in their profile }); if  
(error) throw error; } }
```

3. Local Cache (localStorage)

localStorage is built into every browser. It stores data as key-value pairs and persists even after closing the tab. We use it so the app loads data instantly without waiting for Supabase.

JAVASCRIPT

Cache functions

```
// uid() returns the current user's unique ID – used as a cache key // This ensures each
// user's data is stored separately on the same device const uid = () => user?.id ||
'local'; function loadCache() { // Try to get saved expenses from localStorage (returns
null if empty) expenses = JSON.parse(localStorage.getItem('e_' + uid()) || '[]');
categories = JSON.parse(localStorage.getItem('c_' + uid()) || 'null') || DEF_CATS.map(c
=> ({...c})); // use defaults if none saved } function saveCache() { // Convert data to
JSON string and save to localStorage localStorage.setItem('e_' + uid(),
JSON.stringify(expenses)); localStorage.setItem('c_' + uid(),
JSON.stringify(categories)); }
```

4. Cloud Sync (syncCloud)

This function fetches fresh data from Supabase after login and updates both the local cache and the screen. It runs in the background so the user sees cached data immediately.

JAVASCRIPT

syncCloud() function

```
async function syncCloud() { setPill('sy'); // show "⌚ Syncing..." badge in header try { //
Run two database queries at the same time (Promise.all = parallel, faster) const [{ data:
exps }, { data: cats }] = await Promise.all([ sb.from('expenses') .select('*') // get all
columns .eq('user_id', user.id) // only THIS user's rows .order('date', { ascending:
false }), // newest first sb.from('categories') .select('*') .eq('user_id', user.id) ]);
expenses = exps || []; categories = cats?.length ? cats : await pushDefaultCats();
saveCache(); // update local cache with fresh cloud data renderAll(); // redraw all
charts and lists setPill('ok'); // show "☁ Synced" badge } catch(e) { setPill('er'); //
show "⚠ Error" badge } }
```

5. Adding an Expense

JAVASCRIPT

addExp() function

```

async function addExp() { // 1. Read form values
  const name = document.getElementById('eName').value.trim();
  const amt = parseFloat(document.getElementById('eAmt').value);
  // 2. Validate – stop if empty or invalid
  if (!name || !amt || isNaN(amt)) { toast('Fill required fields', 'err'); return; }
  // 3. Save to Supabase (if connected) and get back the row with its ID
  const { data, error } = await sb.from('expenses').insert([
    { name, amt, cat, date, note, user_id: user.id }
  ]).select();
  // .select() returns the newly inserted row if (error) throw error;
  // 4. Add to local array at the front (unshift = prepend)
  expenses.unshift(data[0]);
  saveCache();
  // 5. Update localStorage // 6. Check if this pushes a category over budget
  checkBudgetAlert(cat, date);
  renderAll();
  // 7. Refresh charts and lists
  toast('Saved ✅', 'ok');
}

```

6. Budget Alert Logic

JAVASCRIPT

checkBudgetAlert() function

```

function checkBudgetAlert(cat, date) {
  const budget = catBudget(cat); // get this category's limit
  if (!budget) return; // no limit set → skip
  // Add up all expenses in this category this month
  const spent = expenses.filter(e => e.cat === cat && e.date?.startsWith(curMonth))
    .reduce((sum, e) => sum + Number(e.amt), 0);
  const pct = (spent / budget) * 100;
  if (pct ≥ 100) toast(`⚠️ ${cat} budget exceeded!`, 'err');
  else if (pct ≥ 80) toast(`🔔 ${cat} at ${Math.round(pct)}% of budget`, 'info');
}

```

7. Rendering Charts (Chart.js)

JAVASCRIPT

renderDonut() function

```

function renderDonut(filteredExpenses) {
  // Group expenses by category and sum amounts
  const bc = {};
  filteredExpenses.forEach(e => { bc[e.cat] = (bc[e.cat] || 0) + Number(e.amt); });
  const labels = Object.keys(bc); // ["Food & Dining", "Transport", ...]
  const data = labels.map(l => bc[l]); // [5200, 800, ...]
  const colors = labels.map(l => cc(l)); // ["#f59e0b", "#3b82f6", ...]
  // Destroy old chart before drawing new one (prevents memory leak)
  if (donutChart) donutChart.destroy();
  // Create a new Chart.js doughnut chart
  donutChart = new Chart(canvas.getContext('2d'), {
    type: 'doughnut',
    data: { labels, datasets: [{ data, backgroundColor: colors, borderWidth: 0 }] },
    options: { cutout: '65%', ... } // cutout makes the hole in the middle
  });
}

```

8. Export to PDF

Uses the **jsPDF** library (loaded from CDN). It creates a blank PDF page, draws text and a table, then triggers a download.

JAVASCRIPT

expPDF() function (simplified)

```
function expPDF() { const { jsPDF } = window.jspdf; const doc = new jsPDF(); // create a blank A4 PDF doc.setFillStyle(15,17,23); // dark background colour (R,G,B) doc.rect(0,0,210,297,'F'); // fill the entire page doc.text('Expense Report', 14, 18); // add text at position x=14, y=18 // autoTable plugin draws the expense rows as a formatted table doc.autoTable({ head: [['Date','Description','Category','Amount']], body: expenses.map(e => [e.date, e.name, e.cat, e.amt]) }); doc.save('expense_report.pdf'); // triggers file download in browser }
```

SECTION 07

UI Screens Explained

SCREEN	HOW TO ACCESS	WHAT IT SHOWS / DOES
Auth Screen	Shown on first open or after logout	Sign In / Sign Up form. Validates email and password. On success, loads the main app.
Dashboard	 bottom nav tab (default)	Period tabs, insight cards (vs last month, avg/day, biggest), stat cards, donut chart, bar chart, budget overview, recent expenses.
Expenses	 bottom nav tab	Full searchable, filterable expense list. Filter by date range, category chip, or text search. Shows filtered total. Export CSV/PDF.
Categories	 bottom nav tab	List of all categories with colour and budget input. Shows how much is spent this month vs the limit. Tap name to rename.
Profile	 bottom nav tab	User info, sync status, all-time stats, backup/restore, delete data, sign out.
Add Expense	+ FAB button (bottom right)	Bottom sheet with form: name, amount, date, category, note. Saves to Supabase instantly.
Edit Expense	 button on any expense row	Same form pre-filled with existing data. Save updates the record in Supabase.
Add Category	+ Add button on Categories header	Name, colour picker, budget amount. New category appears immediately.

SECTION 08

Data Flow

What Happens When You Add an Expense

1 User fills form and taps "Save"

`addExp()` function is called. Inputs are validated.

2 Data sent to Supabase

An INSERT query runs against the expenses table. Supabase creates a row and returns it with the auto-generated id.

3 Added to local array

The returned row is prepended to the `expenses[]` JavaScript array using `unshift()`.

4 Cache updated

`saveCache()` writes the updated array to `localStorage` as a JSON string.

5 Budget check

`checkBudgetAlert()` recalculates the category total and shows a toast if over 80% or 100%.

6 UI re-rendered

`renderAll()` redraws the charts, stat cards, and expense list with the new data.

What Happens on App Open (New Device)

1 Page loads

`SB_URL` and `SB_KEY` are read. Supabase client is created.

2 Session check

`sb.auth.getSession()` checks if there is a saved login token in the browser. If yes → go to step 4.

3 No session → show auth screen

User logs in. `onAuthStateChange` fires `SIGNED_IN` event.

4 loadCache()

Data is loaded from `localStorage` (empty on new device, so shows blank).

5 showApp()

App UI appears with cached (or empty) data.

6 syncCloud()

Fetches all expenses and categories from Supabase. Updates screen with real data. Shows "☁ Synced" badge.

SECTION 09

Deployment Guide

Deploy to GitHub Pages (Free Hosting)

1 Create GitHub Account

Go to github.com → Sign up free.

2 New Repository

Click "+" → New Repository → Name it "expense-tracker" → Set to Public → Click "Create Repository".

3 Upload Files

Click "Add file" → "Upload files" → Upload your index.html and manifest.json → Click "Commit changes".

4 Enable GitHub Pages

Go to Settings → Pages → Source: Deploy from a branch → Branch: main → Folder: / (root) → Save.

5 Your app is live!

URL will be: <https://yourusername.github.io/expense-tracker/>. Takes ~2 minutes to go live.

Install as Android App (PWA)

1 Open Chrome on Android

Go to your GitHub Pages URL.

2 Install prompt

Chrome shows "Add to Home screen" banner at the bottom. Tap it. Or tap the : menu → "Add to Home screen".

3 Done!

App icon appears on your home screen. Opens full-screen, no browser bar. Works on Nothing Phone.

Generate a Real APK (PWABuilder)

1 Go to pwabuilder.com

Paste your GitHub Pages URL → Click Start.

2 Package for Android

Click "Package for Store" → Android → Download the .apk file.

3 Install on phone

Enable "Install unknown apps" in Android Settings → Install the .apk. Done!

SECTION 10

Troubleshooting

PROBLEM	CAUSE	FIX
Login button stays disabled after logout	Button was disabled and never re-enabled	The <code>resetAuthUI()</code> function re-enables it. Refresh the page if stuck.
Data not showing on another browser	Supabase RLS blocking reads	Make sure you ran the RLS policy SQL. Also check you are logged in.
"Sync error" badge shows	Wrong URL or key, or RLS not set	Double-check <code>SB_URL</code> and <code>SB_KEY</code> . Re-run the SQL in Supabase.
Budget overview not showing	No budgets set, or wrong period	Go to  Categories → enter a ₹ amount in the budget field.
Charts not rendering	Chart.js CDN blocked or old chart not destroyed	Check internet connection. The <code>destroy()</code> call before each chart prevents this.
PDF export blank	No expenses match filter	Clear filters first. jsPDF needs at least 1 row to generate a table.
Can't install as PWA	<code>manifest.json</code> missing or HTTPS not used	GitHub Pages uses HTTPS automatically. Make sure <code>manifest.json</code> is in the repo.
New category not saving to cloud	budget column missing from categories table	Run: <code>alter table categories add column if not exists budget numeric default 0;</code>

SECTION 11

Future Enhancements



Push Notifications

Alert the user on their phone when they are nearing a budget limit, even when the app is closed. Uses Web Push API.



Recurring Expenses

Mark expenses like rent or subscriptions as recurring. The app auto-adds them on the set date every month.



Multi-Currency

Support USD, EUR, GBP alongside INR. Auto-convert using live exchange rates from an API.



AI Insights

Use an AI API to analyse spending patterns and suggest savings tips, e.g. "You spent 40% more on food this month."



Receipt Scanner

Take a photo of a receipt and auto-fill the expense form using OCR (Optical Character Recognition).



Shared Accounts

Allow multiple users (e.g. family members) to share one expense pool with individual logins.

SECTION 12

Glossary

TERM	SIMPLE EXPLANATION
Supabase	A free cloud service that gives you a database, login system, and API without writing any backend code.
PostgreSQL	The type of database Supabase uses. Data is stored in tables with rows and columns, like a very powerful Excel.
RLS (Row Level Security)	A Supabase feature that ensures each user can only read and write their own rows — nobody can see another user's data.
localStorage	A storage area built into every browser. Stores data as text that persists even after closing the tab. Limited to ~5 MB.
async / await	JavaScript keywords used when a task takes time (like a network request). await pauses the function until the result arrives.
PWA	Progressive Web App — a website that can be installed on a phone and used like a native app, including offline support.
CDN	Content Delivery Network — servers around the world that host library files (like Chart.js) so you can load them fast without downloading.
REST API	A way for the app to talk to Supabase over the internet using standard HTTP requests (GET, POST, PATCH, DELETE).
JSON	JavaScript Object Notation — a text format for storing structured data. e.g. <code>{"name": "Grocery", "amt": 500}</code>
UUID	Universally Unique Identifier — a long random ID Supabase assigns to every user account, e.g. a1b2c3d4-...
Donut Chart	A circular chart with a hole in the middle. Each slice represents a category's share of total spending.
FAB	Floating Action Button — the + button that floats above the bottom nav. Tapping it opens the Add Expense form.

Bottom Sheet	A panel that slides up from the bottom of the screen. Used for forms (add/edit expense, add category).
Promise.all	Runs multiple async tasks at the same time (in parallel) and waits for all of them. Faster than running them one by one.

Expense Tracker — Advanced Edition

Built with HTML, CSS, Vanilla JavaScript · Backend by Supabase · Charts by Chart.js · PDF by jsPDF
Hosted free on GitHub Pages · Installable as PWA on Android

To save this as a PDF: click the **"Save as PDF"** button at the top, or press **Ctrl+P** (Windows) / **Cmd+P** (Mac) → select "Save as PDF" as the destination.